Politécnico de Tomar – Escola Superior de Tecnologia de Tomar

Linguagem Assembly x86 (16 bits) – Trabalho Prático

Divisão Inteira e Raiz Quadrada

Elementos do grupo:

Guilherme Carril Pereira nº27226

Daniel Filipe Batista Petinga nº27466

Data: 30/10/2026

1. Introdução

Este trabalho pretende implementar em Assembly x86 de 16 bits, no simulador EMU8086, dois algoritmos apresentados em aula: a divisão inteira e a raiz quadrada. Ambos os programas pedem os valores ao utilizador, efetuam os cálculos apenas com as operações básicas (adição, subtração e multiplicação) e com variáveis inteiras, e apresentam no ecrã cada passo parcial e o resultado final.

Nota sobre a instrução DIV: conforme a nota prévia do enunciado, a instrução de divisão inteira é usada apenas para separar e para converter valores binários em decimal na apresentação. O cálculo do quociente e da raiz é feito só com MUL, ADD, SUB e CMP.

2. Tarefa 1.1 – Divisão inteira

2.1 Algoritmo

O algoritmo é a divisão 'por algarismos' da aula: o dividendo é processado da esquerda para a direita. Em cada passo forma-se um valor parcial (resto anterior × 10 + algarismo) e procura-se o maior algarismo q, de 0 a 9, tal que divisor × q ≤ valor parcial. Testam-se os candidatos 1, 2, 3… com multiplicação e comparação; quando o produto ultrapassa o valor parcial, o algarismo anterior é o escolhido. Não há subtrações repetidas.

2.2 Pseudocódigo

```
INÍCIO
  ler dividendo (guardar algarismos em vetor digs[], nº de algarismos n)
  ler divisor
  se divisor = 0 ou divisor > 6553 então ERRO e FIM
  quociente ← 0 ; resto ← 0
  para si ← 0 até n-1 fazer
     atual ← resto × 10 + digs[si]              (MUL, ADD)
     q ← 0
     enquanto (q+1) ≤ 9 e (q+1) × divisor ≤ atual fazer
        q ← q + 1                               (MUL, CMP)
     produto ← q × divisor                      (MUL)
     resto ← atual − produto                    (SUB)
     quociente ← quociente × 10 + q             (MUL, ADD)
     mostrar passo: si+1, digs[si], atual, q, produto, resto
  mostrar quociente e resto
FIM
```

2.3 Fluxograma

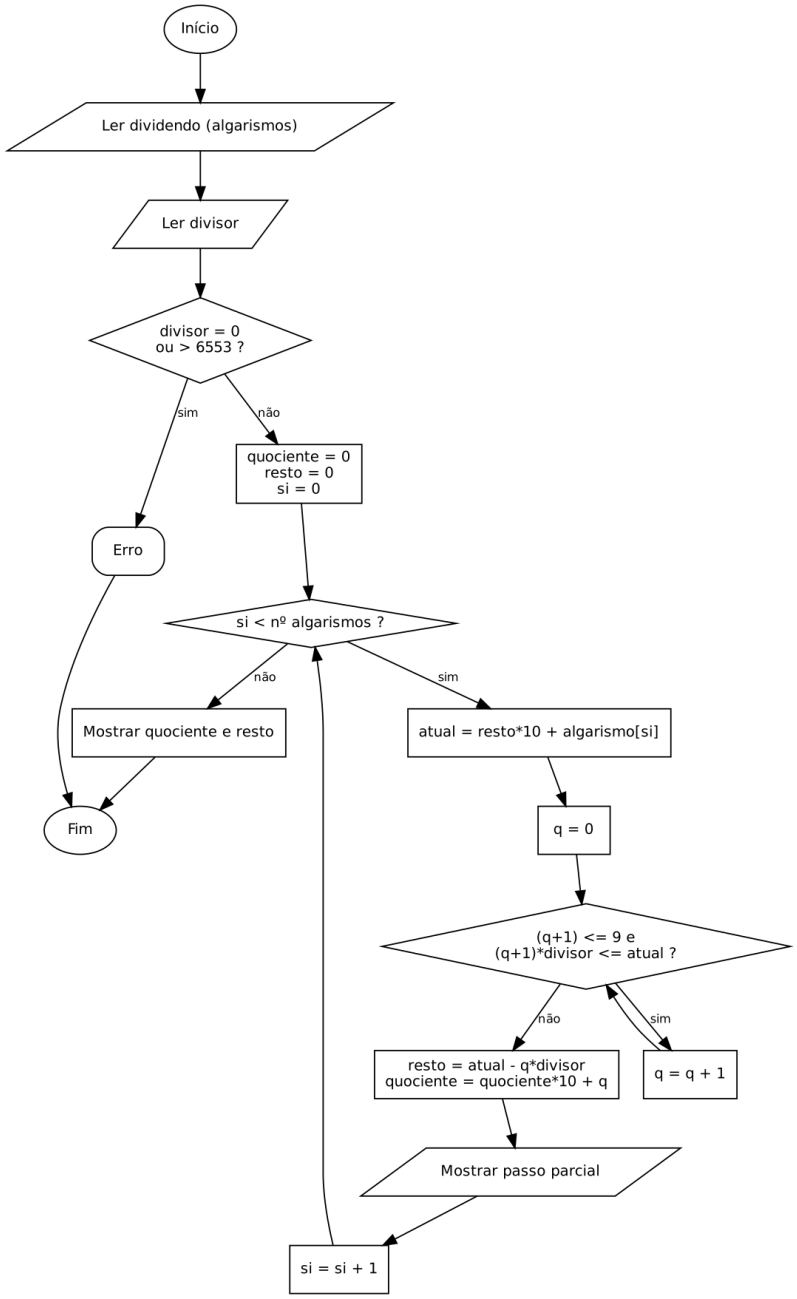


Figura 1 – Fluxograma da divisão inteira.

2.4 Exemplo (do caderno da aula)

Dividendo 73, divisor 25.

Primeiro algarismo 7: atual = 7;

25×1 = 25 > 7, logo q = 0.

Segundo algarismo 3: atual = 73;

25×1 = 25 ≤ 73 ; 25×2 = 50 ≤ 73 ; 25×3 = 75 > 73, logo q = 2, produto = 50 e resto = 73 − 50 = 23.

Resultado: quociente 2, resto 23 (73 = 25 × 2 + 23).

2.5 Testes

O algoritmo foi verificado com um modelo em Python (mesma lógica do programa) para 20 000 pares dividendo/divisor aleatórios (dividendo até 65535, divisor entre 1 e 6553), comparados com a divisão de referência. Não houve diferenças. O programa em Assembly deve ser testado no EMU8086 com os casos: 73/25 (2, resto 23), 65535/6553 (10, resto 5), 0/7 (0, resto 0) e divisor 0 (erro).

3. Tarefa 1.2 – Raiz quadrada

3.1 Algoritmo

O radicando é agrupado em pares de algarismos a partir da direita. Para cada par P, o resto corrente passa a rem × 100 + P. Escolhe-se o maior x (0 a 9) tal que (20 × raiz + x) × x ≤ rem. Depois rem ← rem − (20 × raiz + x) × x e raiz ← raiz × 10 + x. Este é o método apresentado no caderno (com o cálculo de 2 × R × 10 + a) × a).

3.2 Pseudocódigo

```
INÍCIO
  ler radicando (0..9999)  ; se > 9999 então ERRO e FIM
  separar em pares: par1 ← radicando DIV 100 ; par2 ← radicando MOD 100
  rem ← 0 ; raiz ← 0
  para cada par P (par1, par2) fazer
     rem ← rem × 100 + P                          (MUL, ADD)
     x ← 0
     enquanto (x+1) ≤ 9 e (20×raiz + x+1) × (x+1) ≤ rem fazer
        x ← x + 1                                 (MUL, CMP)
     produto ← (20 × raiz + x) × x                (MUL, ADD)
     rem ← rem − produto                          (SUB)
     raiz ← raiz × 10 + x                         (MUL, ADD)
     mostrar passo: nº do par, P, rem, x, produto, raiz
  mostrar raiz e resto
FIM
```

3.3 Fluxograma

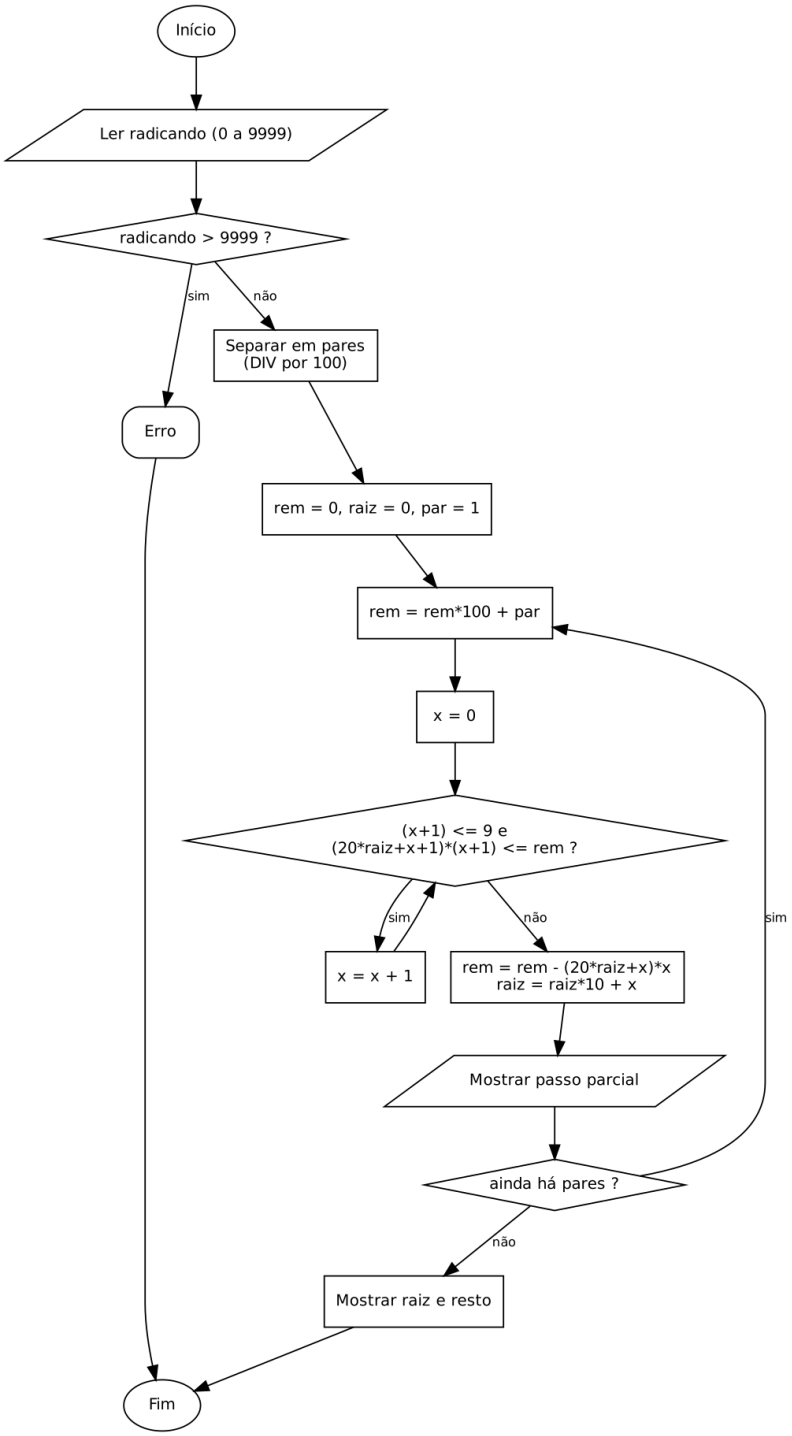


Figura 2 – Fluxograma da raiz quadrada.

3.4 Exemplo

Radicando 341: pares 3 e 41.

Par 3: rem = 3;

x = 1 porque 1×1 = 1 ≤ 3 e 2×2 = 4 > 3;

produto = 1, rem = 2, raiz = 1.

Par 41: rem = 2×100 + 41 = 241;

x = 8 porque 28×8 = 224 ≤ 241 e 29×9 = 261 > 241;

produto = 224, rem = 17, raiz = 18.

Resultado: raiz 18, resto 17 ($18^2 = 324$, 341 − 324 = 17).

3.5 Testes

Com o mesmo modelo Python, verificaram-se todos os radicandos de 0 a 9999 contra math.isqrt, sem diferenças (ex.: 9999 → raiz 99, resto 198).

4. Conclusão

Os dois algoritmos foram implementados seguindo o método da aula. A divisão deixou de usar subtrações sucessivas: o quociente de cada algarismo é encontrado por multiplicações e comparações. A raiz quadrada é calculada par a par, com apresentação de cada passo. Os modelos de verificação confirmam a lógica; a execução final no EMU8086 fica a cargo do grupo, por não ter sido possível compilar o código neste ambiente.

Anexos

Anexo A – Divisão inteira (versão comentada)

```
; ============================================================
; Tarefa 1.1 - Divisao inteira (algoritmo da aula, divisao "por digitos")
; Assembly x86 16 bits - EMU8086 (sintaxe MASM)
;
; Algoritmo:
;   - Os algarismos do dividendo sao lidos um a um.
;   - Para cada algarismo d:  atual = resto * 10 + d
;   - Procura-se o maior q (0..9) tal que  divisor * q <= atual
;     (testa-se q = 1, 2, 3, ... com MUL e CMP; para quando divisor*c > atual)
;   - quociente = quociente * 10 + q
;   - resto     = atual - divisor * q
;   - Os calculos usam apenas MUL, ADD, SUB e CMP (sem subtracoes sucessivas).
;   - A instrucao DIV so e usada na apresentacao (conversao binario -> decimal).
; ============================================================
.model small
```

```
.stack 100h

.data
msgDvd    db 13,10,'Dividendo (0-65535, max 5 dígitos): $'
msgDvs    db 13,10,'Divisor (1-6553): $'
msgErr    db 13,10,'ERRO: divisor invalido (deve ser 1 a 6553).$'
msgCab    db 13,10,'Passos da divisão (digito a digito):',13,10,'$'
msgPasso  db 'Passo $'
msgDig    db ' | algarismo = $'
msgAt     db ' | atual = $'
msgQ      db ' | q = $'
msgPr     db ' | divisor*q = $'
msgRs     db ' | resto = $'
msgQuo    db 13,10,13,10,'Quociente = $'
msgRes    db 13,10,'Resto      = $'
crlf      db 13,10,'$'

digs      db 5 dup(0)          ; algarismos do dividendo
ndig      dw 0                 ; numero de algarismos lidos
dividendo dw 0
divisor   dw 0
quoc      dw 0                 ; quociente acumulado
resto     dw 0                 ; resto acumulado
atual     dw 0                 ; valor parcial (resto*10 + algarismo)
qdig      dw 0                 ; algarismo do quociente
prod      dw 0                 ; divisor * qdig
dtmp      dw 0                 ; algarismo lido (temporário)

.code
main proc
    mov ax,@data                 ; inicializa DS
    mov ds,ax

    lea dx,msgDvd                ; pede o dividendo
    mov ah,9
    int 21h
    call ler_digitos             ; lê algarismos e calcula o valor em AX
    mov dividendo,ax

    lea dx,msgDvs                ; pede o divisor
    mov ah,9
    int 21h
    call ler_valor               ; lê o divisor em AX
    mov divisor,ax

    cmp divisor,0                ; validação do divisor
    je erro
    cmp divisor,6553
    ja erro

    lea dx,msgCab               ; titulo dos passos
    mov ah,9
    int 21h

    mov quoc,0
    mov resto,0
    mov si,0                     ; SI = índice do algarismo corrente

prox_dig:
    cmp si,ndig                  ; processados todos os algarismos?
    jae fim

    ; atual = resto*10 + algarismo
    mov ax,resto
```

```
    mov bx,10
    mul bx                      ; AX = resto*10
    mov bl,digs[si]
    mov bh,0
    add ax,bx                   ; AX = resto*10 + algarismo
    mov atual,ax

    ; procura o maior q (0..9) com divisor*q <= atual
    mov qdig,0
busca:
    mov ax,qdig
    inc ax                      ; candidato c = q + 1
    cmp ax,10
    jae achou                   ; c > 9: para
    mov bx,divisor
    mul bx                      ; AX = c * divisor (DX = 0)
    cmp ax,atual
    ja achou                    ; c * divisor > atual: para
    inc qdig                    ; aceita c como algarismo do quociente
    jmp busca

achou:
    ; prod = qdig * divisor ; resto = atual - prod
    mov ax,qdig
    mov bx,divisor
    mul bx
    mov prod,ax
    mov ax,atual
    sub ax,prod
    mov resto,ax

    ; quociente = quociente*10 + qdig
    mov ax,quoc
    mov bx,10
    mul bx
    add ax,qdig
    mov quoc,ax

    call imprime_passo          ; mostra o passo parcial

    inc si
    jmp prox_dig

fim:
    lea dx,msgQuo               ; resultado final
    mov ah,9
    int 21h
    mov ax,quoc
    call mostrar_numero

    lea dx,msgRes
    mov ah,9
    int 21h
    mov ax,resto
    call mostrar_numero
    jmp sair

erro:
    lea dx,msgErr
    mov ah,9
    int 21h

sair:
    mov ah,4Ch                  ; termina o programa
```

```
    int 21h
main endp

; ------------------------------------------------------------
; ler_digitos: le algarismos ate ENTER.
; Guarda cada algarismo em digs[] e devolve o valor em AX.
; valor = valor*10 + algarismo  (MUL + ADD)
; ------------------------------------------------------------
ler_digitos proc
    push bx
    push cx
    push dx
    xor bx,bx                   ; BX = valor acumulado
    mov ndig,0
ld_loop:
    mov ah,1                    ; lê um caracter com eco
    int 21h
    cmp al,13                   ; ENTER termina
    je ld_fim
    sub al,'0'                  ; caracter -> algarismo
    mov ah,0
    mov dtmp,ax                 ; guarda o algarismo
    mov si,ndig
    cmp si,5                    ; no máximo 5 algarismos
    jae ld_calc
    mov al,byte ptr dtmp
    mov digs[si],al
    inc ndig
ld_calc:
    mov ax,bx
    mov cx,10
    mul cx                      ; AX = valor * 10
    add ax,dtmp                 ; AX = valor*10 + algarismo
    mov bx,ax
    jmp ld_loop
ld_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_digitos endp

; ------------------------------------------------------------
; ler_valor: le um numero (sem guardar algarismos) e devolve em AX
; ------------------------------------------------------------
ler_valor proc
    push bx
    push cx
    push dx
    xor bx,bx
lv_loop:
    mov ah,1
    int 21h
    cmp al,13
    je lv_fim
    sub al,'0'
    mov ah,0
    mov dtmp,ax
    mov ax,bx
    mov cx,10
    mul cx
    add ax,dtmp
    mov bx,ax
```

```
    jmp lv_loop
lv_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_valor endp

; ------------------------------------------------------------
; imprime_passo: mostra o passo corrente (indice SI)
; ------------------------------------------------------------
imprime_passo proc
    push ax
    push dx
    lea dx,msgPasso
    mov ah,9
    int 21h
    mov ax,si
    inc ax                      ; numero do passo = SI + 1
    call mostrar_numero

    lea dx,msgDig
    mov ah,9
    int 21h
    mov al,digs[si]
    mov ah,0
    call mostrar_numero

    lea dx,msgAt
    mov ah,9
    int 21h
    mov ax,atual
    call mostrar_numero

    lea dx,msgQ
    mov ah,9
    int 21h
    mov ax,qdig
    call mostrar_numero

    lea dx,msgPr
    mov ah,9
    int 21h
    mov ax,prod
    call mostrar_numero

    lea dx,msgRs
    mov ah,9
    int 21h
    mov ax,resto
    call mostrar_numero

    lea dx,crlf
    mov ah,9
    int 21h
    pop dx
    pop ax
    ret
imprime_passo endp

; ------------------------------------------------------------
; mostrar_numero: imprime AX (0..65535) em decimal.
; Usa DIV por 10 para obter os algarismos (apenas apresentacao).
```

```
; ------------------------------------------------------------
mostrar_numero proc
    push ax
    push bx
    push cx
    push dx
    mov bx,10
    mov cx,0                    ; contador de algarismos
mn_conv:
    xor dx,dx
    div bx                      ; AX = AX / 10 , DX = resto (algarismo)
    push dx
    inc cx
    cmp ax,0
    jne mn_conv
mn_imp:
    pop dx
    add dl,'0'                  ; algarismo -> ASCII
    mov ah,2
    int 21h
    loop mn_imp
    pop dx
    pop cx
    pop bx
    pop ax
    ret
mostrar_numero endp

end main
```

Anexo B – Divisão inteira (versão sem comentários)

```
.model small
.stack 100h
.data
msgDvd    db 13,10,'Dividendo (0-65535, max 5 digitos): $'
msgDvs    db 13,10,'Divisor (1-6553): $'
msgErr    db 13,10,'ERRO: divisor invalido (deve ser 1 a 6553).$'
msgCab    db 13,10,'Passos da divisao (digito a digito):',13,10,'$'
msgPasso  db 'Passo $'
msgDig    db ' | algarismo = $'
msgAt     db ' | atual = $'
msgQ      db ' | q = $'
msgPr     db ' | divisor*q = $'
msgRs     db ' | resto = $'
msgQuo    db 13,10,13,10,'Quociente = $'
msgRes    db 13,10,'Resto      = $'
crlf      db 13,10,'$'
digs      db 5 dup(0)
ndig      dw 0
dividendo dw 0
divisor   dw 0
quoc      dw 0
resto     dw 0
atual     dw 0
qdig      dw 0
prod      dw 0
dtmp      dw 0
.code
main proc
    mov ax,@data
    mov ds,ax
```

```
    lea dx,msgDvd
    mov ah,9
    int 21h
    call ler_digitos
    mov dividendo,ax
    lea dx,msgDvs
    mov ah,9
    int 21h
    call ler_valor
    mov divisor,ax
    cmp divisor,0
    je erro
    cmp divisor,6553
    ja erro
    lea dx,msgCab
    mov ah,9
    int 21h
    mov quoc,0
    mov resto,0
    mov si,0
prox_dig:
    cmp si,ndig
    jae fim
    mov ax,resto
    mov bx,10
    mul bx
    mov bl,digs[si]
    mov bh,0
    add ax,bx
    mov atual,ax
    mov qdig,0
busca:
    mov ax,qdig
    inc ax
    cmp ax,10
    jae achou
    mov bx,divisor
    mul bx
    cmp ax,atual
    ja achou
    inc qdig
    jmp busca
achou:
    mov ax,qdig
    mov bx,divisor
    mul bx
    mov prod,ax
    mov ax,atual
    sub ax,prod
    mov resto,ax
    mov ax,quoc
    mov bx,10
    mul bx
    add ax,qdig
    mov quoc,ax
    call imprime_passo
    inc si
    jmp prox_dig
fim:
    lea dx,msgQuo
    mov ah,9
    int 21h
    mov ax,quoc
    call mostrar_numero
```

```
    lea dx,msgRes
    mov ah,9
    int 21h
    mov ax,resto
    call mostrar_numero
    jmp sair
erro:
    lea dx,msgErr
    mov ah,9
    int 21h
sair:
    mov ah,4Ch
    int 21h
main endp
ler_digitos proc
    push bx
    push cx
    push dx
    xor bx,bx
    mov ndig,0
ld_loop:
    mov ah,1
    int 21h
    cmp al,13
    je ld_fim
    sub al,'0'
    mov ah,0
    mov dtmp,ax
    mov si,ndig
    cmp si,5
    jae ld_calc
    mov al,byte ptr dtmp
    mov digs[si],al
    inc ndig
ld_calc:
    mov ax,bx
    mov cx,10
    mul cx
    add ax,dtmp
    mov bx,ax
    jmp ld_loop
ld_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_digitos endp
ler_valor proc
    push bx
    push cx
    push dx
    xor bx,bx
lv_loop:
    mov ah,1
    int 21h
    cmp al,13
    je lv_fim
    sub al,'0'
    mov ah,0
    mov dtmp,ax
    mov ax,bx
    mov cx,10
    mul cx
```

```
    add ax,dtmp
    mov bx,ax
    jmp lv_loop
lv_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_valor endp
imprime_passo proc
    push ax
    push dx
    lea dx,msgPasso
    mov ah,9
    int 21h
    mov ax,si
    inc ax
    call mostrar_numero
    lea dx,msgDig
    mov ah,9
    int 21h
    mov al,digs[si]
    mov ah,0
    call mostrar_numero
    lea dx,msgAt
    mov ah,9
    int 21h
    mov ax,atual
    call mostrar_numero
    lea dx,msgQ
    mov ah,9
    int 21h
    mov ax,qdig
    call mostrar_numero
    lea dx,msgPr
    mov ah,9
    int 21h
    mov ax,prod
    call mostrar_numero
    lea dx,msgRs
    mov ah,9
    int 21h
    mov ax,resto
    call mostrar_numero
    lea dx,crlf
    mov ah,9
    int 21h
    pop dx
    pop ax
    ret
imprime_passo endp
mostrar_numero proc
    push ax
    push bx
    push cx
    push dx
    mov bx,10
    mov cx,0
mn_conv:
    xor dx,dx
    div bx
    push dx
    inc cx
```

```
    cmp ax,0
    jne mn_conv
mn_imp:
    pop dx
    add dl,'0'
    mov ah,2
    int 21h
    loop mn_imp
    pop dx
    pop cx
    pop bx
    pop ax
    ret
mostrar_numero endp
end main
```

Anexo C – Raiz quadrada (versão comentada)

```
; ============================================================
; Tarefa 1.2 - Raiz quadrada inteira (algoritmo da aula, por pares)
; Assembly x86 16 bits - EMU8086 (sintaxe MASM)
;
; Algoritmo:
;   - O radicando e agrupado em pares de algarismos (DIV por 100).
;   - Para cada par P:  rem = rem * 100 + P
;   - Procura-se o maior x (0..9) tal que  (20*raiz + x) * x <= rem
;   - rem   = rem - (20*raiz + x) * x
;   - raiz  = raiz * 10 + x
;   - Calculos apenas com MUL, ADD, SUB e CMP.
;   - A instrucao DIV so e usada para separar os pares e na apresentacao.
; Limite: radicando de 0 a 9999 (dois pares).
; ============================================================
.model small
.stack 100h

.data
msgRad    db 13,10,'Radicando (0-9999): $'
msgErr    db 13,10,'ERRO: valor fora do intervalo (0-9999).$'
msgCab    db 13,10,'Passos da raiz quadrada (par a par):',13,10,'$'
msgPasso  db 'Par $'
msgP      db ' | par = $'
msgRem    db ' | rem = $'
msgX      db ' | x = $'
msgProd   db ' | (20*raiz+x)*x = $'
msgRz     db ' | raiz parcial = $'
msgRsto   db ' | resto = $'
msgFinal  db 13,10,13,10,'Raiz quadrada inteira = $'
msgRestoF db 13,10,'Resto final            = $'
crlf      db 13,10,'$'

radicando dw 0
pares     dw 0,0              ; pares[0] = centenas (par 1), pares[1] = unidades (par 2)
rem       dw 0                ; resto corrente
raiz      dw 0                ; raiz parcial
xdig      dw 0                ; algarismo x escolhido
prod      dw 0                ; (20*raiz + x)*x
par       dw 0                ; par corrente

.code
main proc
    mov ax,@data
    mov ds,ax
```

```
    lea dx,msgRad               ; pede o radicando
    mov ah,9
    int 21h
    call ler_valor
    mov radicando,ax

    cmp ax,9999                 ; validacao
    ja erro

    ; separa em pares com DIV: centenas = AX/100 ; unidades = AX mod 100
    xor dx,dx
    mov bx,100
    div bx
    mov pares[0],ax             ; primeiro par
    mov pares[2],dx             ; segundo par

    mov rem,0
    mov raiz,0

    lea dx,msgCab
    mov ah,9
    int 21h

    mov si,0                    ; SI = 0 (primeiro par) ou 2 (segundo par)
prox_par:
    mov ax,pares[si]
    mov par,ax

    ; rem = rem*100 + par  (MUL + ADD)
    mov ax,rem
    mov bx,100
    mul bx
    add ax,par
    mov rem,ax

    ; procura o maior x (0..9) com (20*raiz + x)*x <= rem
    mov xdig,0
busca:
    mov ax,xdig
    inc ax                      ; candidato c = x + 1
    cmp ax,10
    jae achou
    mov bx,raiz
    mov cx,ax                   ; CX = c
    mov ax,raiz
    mov bx,20
    mul bx                      ; AX = 20*raiz
    add ax,cx                   ; AX = 20*raiz + c
    mul cx                      ; AX = (20*raiz + c) * c   (DX = 0)
    cmp ax,rem
    ja achou                    ; ultrapassou rem: para
    inc xdig                    ; aceita c
    jmp busca

achou:
    ; prod = (20*raiz + x) * x
    mov ax,raiz
    mov bx,20
    mul bx
    add ax,xdig
    mov bx,xdig
    mul bx
    mov prod,ax
```

```
    ; rem = rem - prod
    mov ax,rem
    sub ax,prod
    mov rem,ax

    ; raiz = raiz*10 + x
    mov ax,raiz
    mov bx,10
    mul bx
    add ax,xdig
    mov raiz,ax

    call imprime_passo

    add si,2                    ; proximo par
    cmp si,4
    jb prox_par

    ; resultado final
    lea dx,msgFinal
    mov ah,9
    int 21h
    mov ax,raiz
    call mostrar_numero

    lea dx,msgRestoF
    mov ah,9
    int 21h
    mov ax,rem
    call mostrar_numero
    jmp sair

erro:
    lea dx,msgErr
    mov ah,9
    int 21h

sair:
    mov ah,4Ch
    int 21h
main endp

; ------------------------------------------------------------
; ler_valor: le digitos ate ENTER e devolve o valor em AX
; valor = valor*10 + algarismo  (MUL + ADD)
; ------------------------------------------------------------
ler_valor proc
    push bx
    push cx
    push dx
    xor bx,bx
lv_loop:
    mov ah,1
    int 21h
    cmp al,13
    je lv_fim
    sub al,'0'
    mov ah,0
    mov cx,ax                   ; CX = algarismo
    mov ax,bx
    mov dx,10
    mul dx                      ; AX = valor*10
    add ax,cx
```

```
    mov bx,ax
    jmp lv_loop
lv_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_valor endp

; ------------------------------------------------------------
; imprime_passo: mostra o passo corrente (par em PAR, etc.)
; ------------------------------------------------------------
imprime_passo proc
    push ax
    push dx
    lea dx,msgPasso
    mov ah,9
    int 21h
    mov ax,si
    shr ax,1
    inc ax                      ; numero do par: 1 ou 2
    call mostrar_numero

    lea dx,msgP
    mov ah,9
    int 21h
    mov ax,par
    call mostrar_numero

    lea dx,msgRem
    mov ah,9
    int 21h
    mov ax,rem
    call mostrar_numero

    lea dx,msgX
    mov ah,9
    int 21h
    mov ax,xdig
    call mostrar_numero

    lea dx,msgProd
    mov ah,9
    int 21h
    mov ax,prod
    call mostrar_numero

    lea dx,msgRz
    mov ah,9
    int 21h
    mov ax,raiz
    call mostrar_numero

    lea dx,crlf
    mov ah,9
    int 21h
    pop dx
    pop ax
    ret
imprime_passo endp

; ------------------------------------------------------------
; mostrar_numero: imprime AX (0..65535) em decimal (DIV por 10)
```

```
; -------------------------------------------------------------
mostrar_numero proc
    push ax
    push bx
    push cx
    push dx
    mov bx,10
    mov cx,0
mn_conv:
    xor dx,dx
    div bx
    push dx
    inc cx
    cmp ax,0
    jne mn_conv
mn_imp:
    pop dx
    add dl,'0'
    mov ah,2
    int 21h
    loop mn_imp
    pop dx
    pop cx
    pop bx
    pop ax
    ret
mostrar_numero endp

end main
```

Anexo D – Raiz quadrada (versão sem comentários)

```
.model small
.stack 100h
.data
msgRad    db 13,10,'Radicando (0-9999): $'
msgErr    db 13,10,'ERRO: valor fora do intervalo (0-9999).$'
msgCab    db 13,10,'Passos da raiz quadrada (par a par):',13,10,'$'
msgPasso  db 'Par $'
msgP      db ' | par = $'
msgRem    db ' | rem = $'
msgX      db ' | x = $'
msgProd   db ' | (20*raiz+x)*x = $'
msgRz     db ' | raiz parcial = $'
msgRsto   db ' | resto = $'
msgFinal  db 13,10,13,10,'Raiz quadrada inteira = $'
msgRestoF db 13,10,'Resto final            = $'
crlf      db 13,10,'$'
radicando dw 0
pares     dw 0,0
rem       dw 0
raiz      dw 0
xdig      dw 0
prod      dw 0
par       dw 0
.code
main proc
    mov ax,@data
    mov ds,ax
    lea dx,msgRad
    mov ah,9
    int 21h
```

```
    call ler_valor
    mov radicando,ax
    cmp ax,9999
    ja erro
    xor dx,dx
    mov bx,100
    div bx
    mov pares[0],ax
    mov pares[2],dx
    mov rem,0
    mov raiz,0
    lea dx,msgCab
    mov ah,9
    int 21h
    mov si,0
prox_par:
    mov ax,pares[si]
    mov par,ax
    mov ax,rem
    mov bx,100
    mul bx
    add ax,par
    mov rem,ax
    mov xdig,0
busca:
    mov ax,xdig
    inc ax
    cmp ax,10
    jae achou
    mov bx,raiz
    mov cx,ax
    mov ax,raiz
    mov bx,20
    mul bx
    add ax,cx
    mul cx
    cmp ax,rem
    ja achou
    inc xdig
    jmp busca
achou:
    mov ax,raiz
    mov bx,20
    mul bx
    add ax,xdig
    mov bx,xdig
    mul bx
    mov prod,ax
    mov ax,rem
    sub ax,prod
    mov rem,ax
    mov ax,raiz
    mov bx,10
    mul bx
    add ax,xdig
    mov raiz,ax
    call imprime_passo
    add si,2
    cmp si,4
    jb prox_par
    lea dx,msgFinal
    mov ah,9
    int 21h
    mov ax,raiz
```

```
    call mostrar_numero
    lea dx,msgRestoF
    mov ah,9
    int 21h
    mov ax,rem
    call mostrar_numero
    jmp sair
erro:
    lea dx,msgErr
    mov ah,9
    int 21h
sair:
    mov ah,4Ch
    int 21h
main endp
ler_valor proc
    push bx
    push cx
    push dx
    xor bx,bx
lv_loop:
    mov ah,1
    int 21h
    cmp al,13
    je lv_fim
    sub al,'0'
    mov ah,0
    mov cx,ax
    mov ax,bx
    mov dx,10
    mul dx
    add ax,cx
    mov bx,ax
    jmp lv_loop
lv_fim:
    mov ax,bx
    pop dx
    pop cx
    pop bx
    ret
ler_valor endp
imprime_passo proc
    push ax
    push dx
    lea dx,msgPasso
    mov ah,9
    int 21h
    mov ax,si
    shr ax,1
    inc ax
    call mostrar_numero
    lea dx,msgP
    mov ah,9
    int 21h
    mov ax,par
    call mostrar_numero
    lea dx,msgRem
    mov ah,9
    int 21h
    mov ax,rem
    call mostrar_numero
    lea dx,msgX
    mov ah,9
    int 21h
```

```
    mov ax,xdig
    call mostrar_numero
    lea dx,msgProd
    mov ah,9
    int 21h
    mov ax,prod
    call mostrar_numero
    lea dx,msgRz
    mov ah,9
    int 21h
    mov ax,raiz
    call mostrar_numero
    lea dx,crlf
    mov ah,9
    int 21h
    pop dx
    pop ax
    ret
imprime_passo endp
mostrar_numero proc
    push ax
    push bx
    push cx
    push dx
    mov bx,10
    mov cx,0
mn_conv:
    xor dx,dx
    div bx
    push dx
    inc cx
    cmp ax,0
    jne mn_conv
mn_imp:
    pop dx
    add dl,'0'
    mov ah,2
    int 21h
    loop mn_imp
    pop dx
    pop cx
    pop bx
    pop ax
    ret
mostrar_numero endp
end main
```